

LAPORAN PRAKTIKUM 8 STRUKTUR DATA
IMPLEMENTASI ALGORITMA SORTING

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah struktur data yang membahas tentang Sorting di Java Eclipse. Dalam praktikum ini, penulis tidak hanya belajar tentang Penyusunan laporan ini bertujuan untuk memahami konsep dasar Sorting.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
1.3 Manfaat.....	2
BAB II	3
PEMBAHASAN	3
2.1 Dasar Teori	3
2.2 Langkah kerja	4
2.2.1 Buka eclipse	4
2.2.2 Membuat Java Project	4
2.2.3 Class 1 “ShellSort_2511531020”	5
2.2.4 Class 2 “Quicksort_2511531020”	9
2.2.5 Class 3 “MergeSort_2511531020”	12
2.2.6 Class 4 “BubleSortGUI_2511531020”	16
2.3 Tugas Praktikum	22
2.3.1 Class 1 “Sorting_2511531020”.....	22
BAB III	30
KESIMPULAN.....	30
DAFTAR PUSTAKA	31

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, algoritma pengurutan (sorting) merupakan salah satu topik yang sangat penting karena hampir semua aplikasi membutuhkan proses pengurutan data. Data yang terurut akan mempermudah proses pencarian, analisis, maupun pengolahan lebih lanjut. Oleh karena itu, pemahaman terhadap berbagai algoritma sorting menjadi hal yang wajib bagi mahasiswa informatika.

Pada praktikum ini, beberapa algoritma sorting diimplementasikan menggunakan bahasa pemrograman Java, yaitu Quick Sort, Merge Sort, Shell Sort, serta Bubble/Insertion Sort dengan tampilan GUI. Selain itu, terdapat penerapan algoritma sorting pada data non-numerik berupa playlist lagu. Dengan adanya variasi implementasi ini, mahasiswa dapat melihat bagaimana algoritma bekerja baik pada data angka maupun teks, serta bagaimana visualisasi dapat membantu memahami langkah-langkah sorting.

1.2 Tujuan

1. Memahami prinsip kerja berbagai algoritma.
2. Menganalisis perbedaan efisiensi antar algoritma.
3. Mengimplementasikan algoritma sorting dalam bahasa Java dengan variasi data.
4. Membuat visualisasi GUI untuk memperjelas proses sorting langkah demi langkah.
5. Menerapkan sorting pada data non-numerik (judul lagu) untuk memperluas pemahaman.

1.3 Manfaat

1. Memberikan pemahaman praktis tentang cara kerja algoritma sorting.
2. Melatih keterampilan pemrograman Java dengan penerapan algoritma.
3. Membantu mahasiswa memahami perbedaan kompleksitas waktu antar algoritma.
4. Menunjukkan aplikasi nyata sorting pada data teks maupun numerik.
5. Meningkatkan kemampuan analisis dan debugging program.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Quick Sort Algoritma berbasis *divide and conquer* dengan memilih pivot, kemudian membagi array menjadi dua bagian. Pada program ini digunakan teknik Median-of-Three untuk memilih pivot agar lebih stabil.

Merge Sort Algoritma rekursif yang membagi array menjadi dua bagian, mengurutkan masing-masing, lalu menggabungkannya kembali. Merge Sort memiliki kompleksitas waktu $O(n \log n)$.

Shell Sort Merupakan pengembangan dari Insertion Sort dengan menggunakan gap tertentu untuk mempercepat proses pengurutan.

Bubble/Insertion Sort GUI Program GUI menggunakan Java Swing untuk menampilkan proses sorting langkah demi langkah. Hal ini membantu visualisasi bagaimana elemen berpindah posisi.

Sorting Lagu (Shell, Quick, Merge)

- Quick Sort digunakan untuk mengurutkan berdasarkan durasi (ascending).
- Merge Sort digunakan untuk mengurutkan judul lagu secara alfabetis.
- Shell Sort juga digunakan untuk mengurutkan judul lagu A-Z.

2.2 Langkah kerja

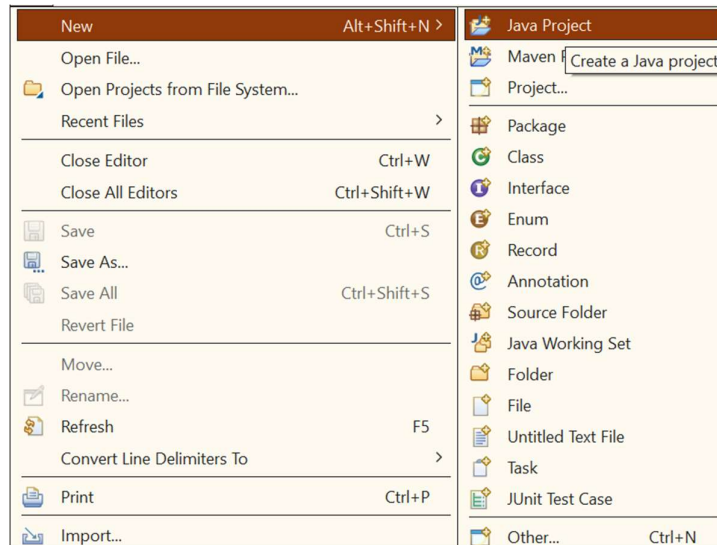
2.2.1 Buka eclipse



Gambar 2.1 Buka eclipse jalankan

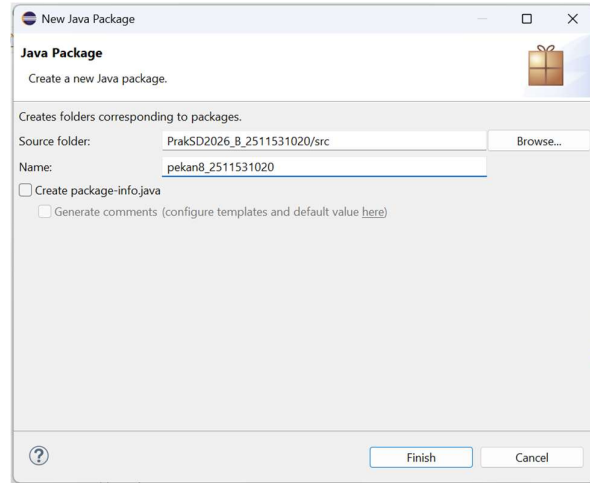
2.2.2 Membuat Java Project

1. Klik file
2. Klik java project



Gambar 2.2 Java Project

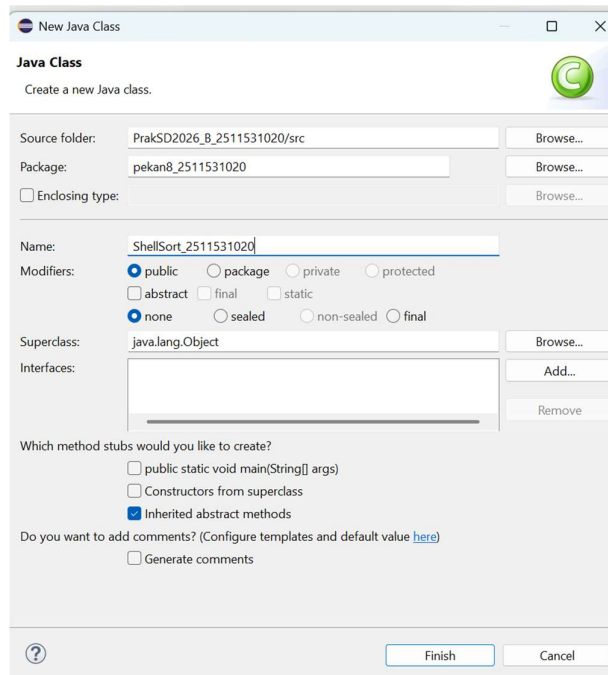
3. Buat nama project



Gambar 2.3 Nama Project

2.2.3 Class 1 “ShellSort_2511531020”

1. Membuat package pekan8_2511531020 dan class ShellSort_2511531020



Gambar 2.4 Membuat Nama package dan class
ShellSort_2511531020

2. Klik Finish

3. Kode Program

```
4. package pekan8_2511531020;
5.
6. public class ShellSort_2511531020 {
7.
8.     public static void shellSort_1020(int[] A_1020) {
9.         int n_1020 = A_1020.length;
10.        int gap_1020 = n_1020 / 2;
11.
12.        while (gap_1020 > 0) {
13.            for (int i_1020 = gap_1020; i_1020 < n_1020;
14.                i_1020++) {
15.                int temp_1020 = A_1020[i_1020];
16.                int j_1020 = i_1020;
17.                while (j_1020 >= gap_1020 &&
18.                    A_1020[j_1020 - gap_1020] > temp_1020) {
19.                    A_1020[j_1020] = A_1020[j_1020 -
20.                    gap_1020];
21.                    j_1020 = j_1020 - gap_1020;
22.                }
23.                A_1020[j_1020] = temp_1020;
24.            }
25.            gap_1020 = gap_1020 / 2;
26.        }
27.    }
28.
29.    public static void main(String[] args) {
30.        int[] data_1020 = {3, 10, 4, 6, 8, 9, 7, 2, 1,
31.        5};
32.        System.out.print("Sebelum: ");
33.        printArray(data_1020);
34.
35.        shellSort_1020(data_1020);
36.
37.        System.out.print("Sesudah (Shell Sort): ");
38.        printArray(data_1020);
39.    }
40.
41.    public static void printArray(int[] arr) {
42.        for (int i_1020 : arr)
43.            System.out.print(i_1020 + " ");
44.
45.        System.out.println();
46.    }
47. }
```

Gambar 2.5 Class 1 “ShellSort_2511531020”

4. Analisis Langkah

a. Struktur Program

- Kelas ShellSort_2511531020 berada dalam package pekan8_2511531020.

b. Elemen sintaks utama:

- shellSort_1020 → metode untuk mengurutkan array dengan algoritma Shell Sort. o main → titik masuk program, berisi array contoh dan pemanggilan fungsi Shell Sort.
- printArray → metode untuk mencetak isi array ke konsol.
- Variabel penting: o A_1020 → array yang akan diurutkan. o n_1020 → panjang array.
- gap_1020 → jarak antar elemen yang dibandingkan.
- i_1020, j_1020 → indeks iterasi.
- temp_1020 → elemen sementara yang disimpan saat proses pergeseran.
- data_1020 → array contoh yang diurutkan di main.

c. Alur Kerja Sintaks

- main membuat array data_1020 berisi angka acak.
- printArray menampilkan array sebelum diurutkan.
- shellSort_1020 dipanggil untuk mengurutkan array:
- Hitung panjang array (n_1020). o Tentukan nilai awal gap ($\text{gap_1020} = \text{n_1020} / 2$).
- Selama $\text{gap_1020} > 0$, lakukan iterasi: – Ambil elemen temp_1020 pada posisi i_1020.
– Bandingkan dengan elemen yang berjarak gap_1020.

- Geser elemen yang lebih besar ke kanan.
- Tempatkan temp_1020 di posisi yang sesuai.
- Kurangi gap menjadi setengah ($\text{gap_1020} = \text{gap_1020} / 2$).
- Setelah selesai, printArray menampilkan array hasil pengurutan.

d. Output Program

- Output berupa teks di konsol: o "Sebelum: ..." → menampilkan array awal.
- "Sesudah (Shell Sort): ..." → menampilkan array hasil Shell Sort.

5. Output



```

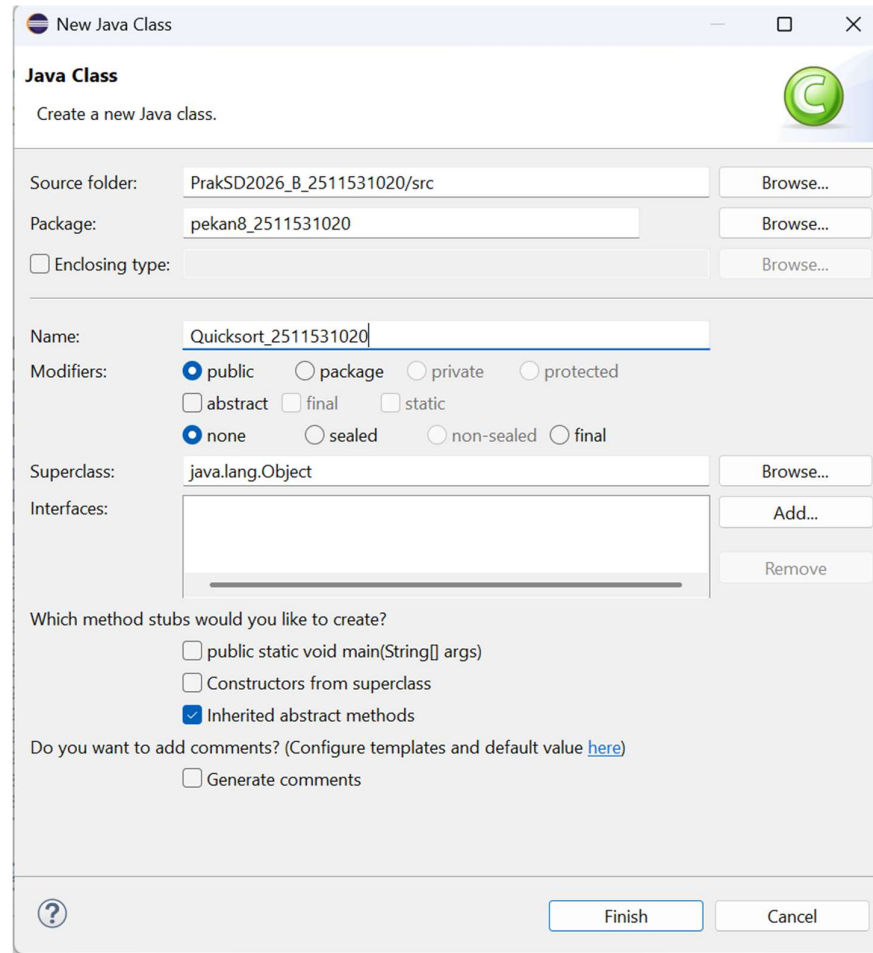
<terminated> ShellSort_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win:
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

```

Gambar 2.6 Output

2.2.4 Class 2 “Quicksort_2511531020”

1. Membuat package pekan8_2511531020 dan class Quicksort_2511531020



Gambar 2.7 Membuat Nama package dan class Quicksort_2511531020

2. Klik Finish
3. Kode Program
4. **package** pekan8_2511531020;
- 5.
6. **public class** Quicksort_2511531020 {
- 7.
8. **static void** swap_2511531020(**int**[] arr_1020, **int** i_1020, **int** j_1020) {
9. **int** temp_1020 = arr_1020[i_1020];
10. arr_1020[i_1020] = arr_1020[j_1020];
11. arr_1020[j_1020] = temp_1020;
12. }
- 13.

```

14. // Metode tambahan untuk mengatur pivot menggunakan Median-
    of-Three
15. static void medianOfThree_2511531020(int[] arr_1020, int
    low_1020, int high_1020) {
16.     int mid_1020 = low_1020 + (high_1020 - low_1020) / 2;
17.
18.     // Urutkan elemen low, mid, dan high
19.     if (arr_1020[low_1020] > arr_1020[mid_1020]) {
20.         swap_2511531020(arr_1020, low_1020, mid_1020);
21.     }
22.
23.     if (arr_1020[low_1020] > arr_1020[high_1020]) {
24.         swap_2511531020(arr_1020, low_1020, high_1020);
25.     }
26.
27.     if (arr_1020[mid_1020] > arr_1020[high_1020]) {
28.         swap_2511531020(arr_1020, mid_1020, high_1020);
29.     }
30.
31.     swap_2511531020(arr_1020, mid_1020, high_1020);
32. }
33.
34. static int partition_2511531020(int[] arr_1020, int low_1020,
    int high_1020) {
35.     // Panggil fungsi medianOfThree sebelum menentukan pivot
36.     medianOfThree_2511531020(arr_1020, low_1020, high_1020);
37.
38.     int pivot_1020 = arr_1020[high_1020]; // Sekarang
    arr[high] sudah berisi nilai median
39.     int i_1020 = (low_1020 - 1);
40.
41.     for (int j_1020 = low_1020; j_1020 <= high_1020 - 1;
        j_1020++) {
42.         // Jika elemen saat ini lebih kecil dari atau sama
        dengan pivot
43.         if (arr_1020[j_1020] <= pivot_1020) {
44.             i_1020++; // Increment indeks elemen yang lebih
            kecil
45.             swap_2511531020(arr_1020, i_1020, j_1020);
46.         }
47.     }
48.
49.     swap_2511531020(arr_1020, i_1020 + 1, high_1020);
50.     return (i_1020 + 1);
51. }
52.
53. static void quickSort_2511531020(int[] arr_1020, int
    low_1020, int high_1020) {
54.     if (low_1020 < high_1020) {
55.         int pi_1020 = partition_2511531020(arr_1020,
            low_1020, high_1020);
56.         quickSort_2511531020(arr_1020, low_1020, pi_1020 -
            1);
57.         quickSort_2511531020(arr_1020, pi_1020 + 1,
            high_1020);
58.     }
59. }

```

```

60.
61.     public static void printArr_2511531020(int[] arr_1020) {
62.         for (int i_1020 = 0; i_1020 < arr_1020.length; i_1020++)
63.         {
64.             System.out.print(arr_1020[i_1020] + " ");
65.         }
66.         System.out.println();
67.     }
68.     public static void main(String[] args) {
69.         int[] arr_1020 = { 10, 7, 8, 9, 1, 5 };
70.         int N_1020 = arr_1020.length;
71.         System.out.print("Data sebelum diurutkan: ");
72.         printArr_2511531020(arr_1020);
73.
74.         quickSort_2511531020(arr_1020, 0, N_1020 - 1);
75.
76.         System.out.print("Data Terurut quicksort: ");
77.         printArr_2511531020(arr_1020);
78.     }
79. }

```

Gambar 2.8 Class 2 “Quicksort_2511531020”

4. Analisis Langkah

a. Struktur Program

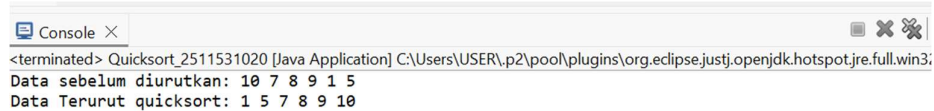
- Kelas Quicksort_2511531020 berisi implementasi algoritma QuickSort dengan metode median-of-three.
- Elemen sintaks utama: o swap_2511531020 → fungsi untuk menukar dua elemen array.
- medianOfThree_2511531020 → fungsi untuk memilih pivot dengan cara mengambil median dari tiga elemen (low, mid, high).
- partition_2511531020 → fungsi untuk membagi array berdasarkan pivot.
- quickSort_2511531020 → fungsi rekursif untuk mengurutkan array.
- printArr_2511531020 → fungsi untuk mencetak isi array.
- main → titik masuk program, berisi array contoh dan pemanggilan fungsi QuickSort.

b. Alur Kerja Sintaks

- main membuat array arr_1020 berisi data acak.
- printArr_2511531020 menampilkan array sebelum diurutkan.

- quickSort_2511531020 dipanggil dengan parameter indeks awal dan akhir.
 - Di dalam quickSort_2511531020:
 - partition_2511531020 dipanggil untuk menentukan posisi pivot.
 - Array dibagi menjadi dua bagian (kiri dan kanan pivot).
 - Fungsi dipanggil kembali secara rekursif untuk kedua bagian.
 - partition_2511531020:
 - Memanggil medianOfThree_2511531020 untuk memilih pivot.
 - Menggeser elemen yang lebih kecil ke kiri.
 - Menempatkan pivot di posisi yang tepat.
 - swap_2511531020 digunakan setiap kali ada pertukaran elemen.
 - Setelah selesai, printArr_2511531020 menampilkan array yang sudah terurut.
- c. Output Program
- Output berupa teks di konsol:
 - "Data sebelum diurutkan: ..." → menampilkan array awal.
 - "Data Terurut quicksort: ..." → menampilkan array hasil QuickSort.

5. Output

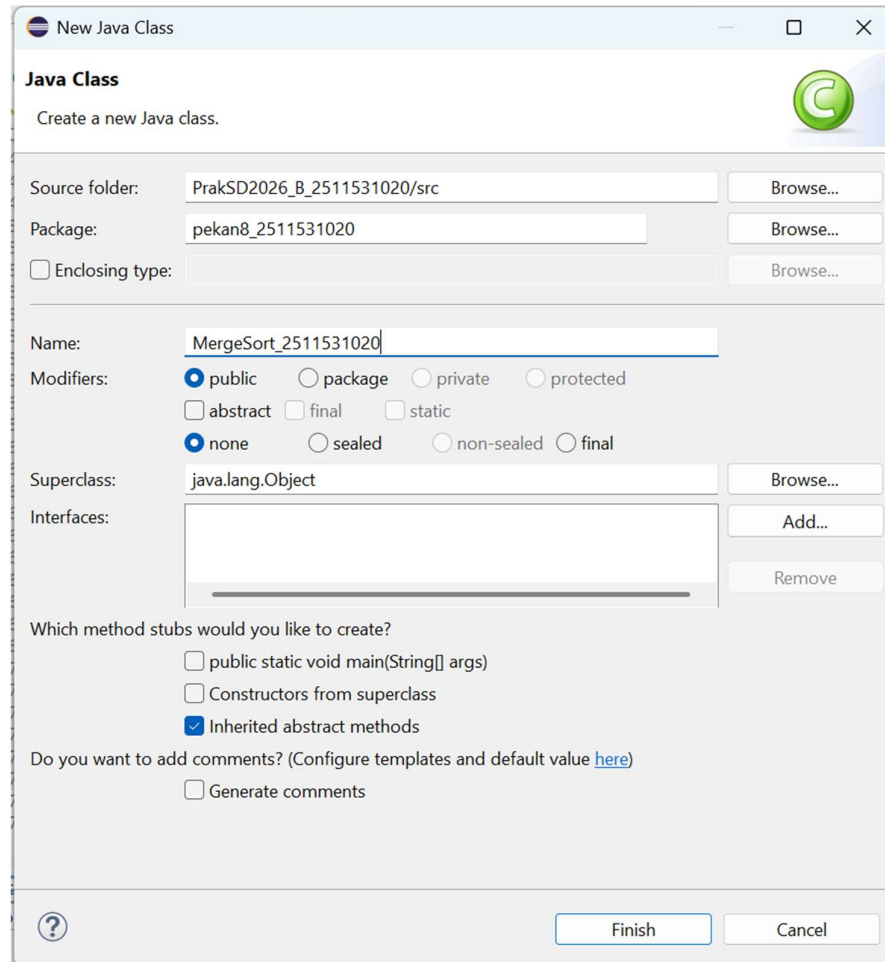


```
<terminated> Quicksort_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32;
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

Gambar 2.9 Output

2.2.5 Class 3 “MergeSort_2511531020”

1. Membuat package pekan8_2511531020 dan class MergeSort_2511531020



Gambar 2.10 Membuat Nama package dan class 3
MergeSort_2511531020

2. Klik Finish

3. Kode Program

```

4. package pekan8_2511531020;
5.
6. public class MergeSort_2511531020 {
7.     void merge_2511531020(int arr_1020[], int l_1020, int
m_1020, int r_1020) {
8.         int n1_1020 = m_1020 - l_1020 + 1;
9.         int n2_1020 = r_1020 - m_1020;
10.
11.         int L_1020[] = new int[n1_1020];
12.         int R_1020[] = new int[n2_1020];
13.
14.         for (int i_1020 = 0; i_1020 < n1_1020; ++i_1020)
15.             L_1020[i_1020] = arr_1020[l_1020 + i_1020];

```

```

16.         for (int j_1020 = 0; j_1020 < n2_1020; ++j_1020)
17.             R_1020[j_1020] = arr_1020[m_1020 + 1 + j_1020];
18.
19.         int i_1020 = 0, j_1020 = 0;
20.         int k_1020 = l_1020;
21.         while (i_1020 < n1_1020 && j_1020 < n2_1020) {
22.             if (L_1020[i_1020] <= R_1020[j_1020]) {
23.                 arr_1020[k_1020] = L_1020[i_1020];
24.                 i_1020++;
25.             } else {
26.                 arr_1020[k_1020] = R_1020[j_1020];
27.                 j_1020++;
28.             }
29.             k_1020++;
30.         }
31.
32.         while (i_1020 < n1_1020) {
33.             arr_1020[k_1020] = L_1020[i_1020];
34.             i_1020++;
35.             k_1020++;
36.         }
37.
38.         while (j_1020 < n2_1020) {
39.             arr_1020[k_1020] = R_1020[j_1020];
40.             j_1020++;
41.             k_1020++;
42.         }
43.     }
44.
45.     // Tambahan dari foto: method sort
46.     void sort_2511531020(int arr_1020[], int l_1020, int r_1020)
47.     {
48.         if (l_1020 < r_1020) {
49.             int m_1020 = (l_1020 + r_1020) / 2;
50.             sort_2511531020(arr_1020, l_1020, m_1020);
51.             sort_2511531020(arr_1020, m_1020 + 1, r_1020);
52.             merge_2511531020(arr_1020, l_1020, m_1020, r_1020);
53.         }
54.
55.         // Utility untuk print array
56.         static void printArray_2511531020(int arr_1020[]) {
57.             int n_1020 = arr_1020.length;
58.             for (int i_1020 = 0; i_1020 < n_1020; ++i_1020)
59.                 System.out.print(arr_1020[i_1020] + " ");
60.             System.out.println();
61.         }
62.
63.         // Main program
64.         public static void main(String args_1020[]) {
65.             int arr_1020[] = { 12, 11, 13, 5, 6, 7 };
66.             System.out.println("Sebelum terurut:");
67.             printArray_2511531020(arr_1020);
68.
69.             MergeSort_2511531020 ob_1020 = new
MergeSort_2511531020();

```

```

70.         ob_1020.sort_2511531020(arr_1020, 0, arr_1020.length -
71.         1);
72.         System.out.println("\nSesudah terurut menggunakan Merge
73.         Sort:");
74.         printArray_2511531020(arr_1020);
75.     }

```

Gambar 2.11 Class 3 “MergeSort_2511531020”

4. Analisis Langkah

a. Struktur Program

- Program berada dalam package pekan8_2511531020.
- Kelas utama bernama MergeSort_2511531020.
- Terdapat tiga metode utama: merge_2511531020, sort_2511531020, dan printArray_2511531020.
- Fungsi main digunakan untuk menjalankan proses pengurutan dan menampilkan hasil.

b. Fungsi merge_2511531020

- Membagi array menjadi dua subarray: kiri (L_1020) dan kanan (R_1020).
- Mengisi masing-masing subarray dengan elemen dari array utama.
- Menggabungkan kembali kedua subarray dengan membandingkan elemen satu per satu.
- Menyalin sisa elemen jika salah satu subarray masih memiliki data.

c. Fungsi sort_2511531020

- Menggunakan rekursi untuk membagi array menjadi dua bagian hingga ukuran terkecil.
- Memanggil dirinya sendiri untuk bagian kiri dan kanan.

- Setelah pembagian selesai, memanggil merge_2511531020 untuk menggabungkan hasilnya.

d. Alur Eksekusi Program

- Array awal {12, 11, 13, 5, 6, 7} ditampilkan sebagai kondisi sebelum terurut.
- Objek MergeSort_2511531020 dibuat untuk memanggil metode sort_2511531020.
- Proses rekursi membagi array menjadi bagian kecil, lalu digabungkan kembali dengan urutan yang benar.
- Setelah semua langkah selesai, array sudah dalam keadaan terurut.
- Hasil akhir ditampilkan di console: 5 6 7 11 12 13.

5. Output



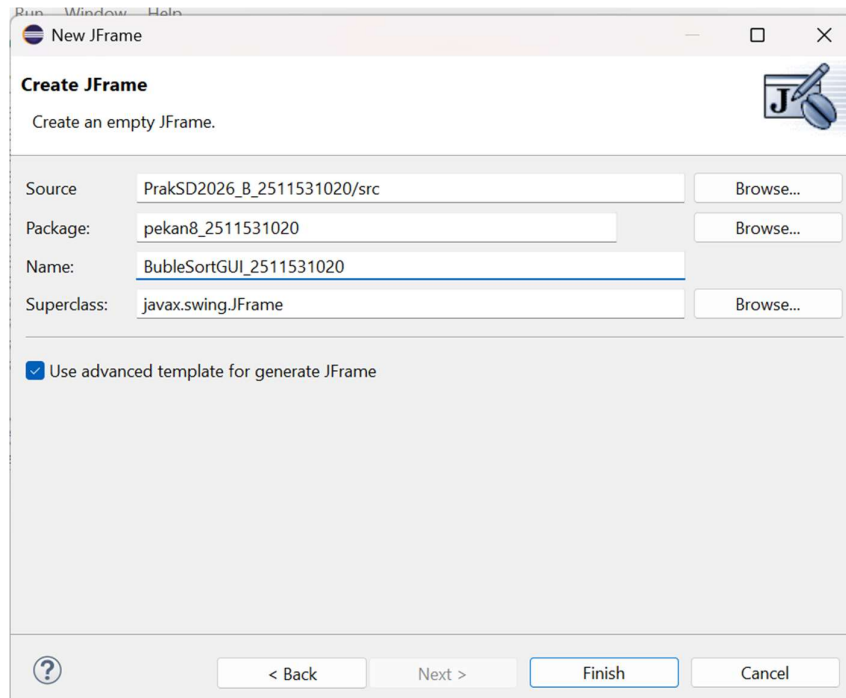
```
<terminated> MergeSort_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.wir
Sebelum terurut:
12 11 13 5 6 7

Sesudah terurut menggunakan Merge Sort:
5 6 7 11 12 13
```

Gambar 2.12 Output

2.2.6 Class 4 “BubleSortGUI_2511531020”

1. Membuat package pekan8_2511531020 dan class BubleSortGUI_2511531020



Gambar 2.13 Class 4 “InsertionSortGUI_2511531020”

2. Klik Finish
3. Kode Program

```

1 package pekan8_2511531020;
2
3 import java.awt.BorderLayout;
19
20 public class BubleSortGUI_2511531020 extends JFrame {
21
22     private static final long serialVersionUID = 1L;
23
24     private int[] array_1020;
25     private JLabel[] labelArray_1020;
26     private JButton stepButton_1020, resetButton_1020, setButton_1020;
27     private JTextField inputField_1020;
28     private JPanel panelArray_1020;
29     private JTextArea stepArea_1020;
30
31     private int i_1020 = 1, j_1020;
32     private boolean sorting_1020 = false;
33     private int stepCount_1020 = 1;
34
35     public BubleSortGUI_2511531020() {
36         setTitle("Insertion Sort: Langkah per Langkah");
37         setSize(750, 400);
38         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
39         setLocationRelativeTo(null);
40         setLayout(new BorderLayout());
41
42         // Panel input
43         JPanel inputPanel_1020 = new JPanel(new FlowLayout());
44         inputField_1020 = new JTextField(30);
45         setButton_1020 = new JButton("Set Array");
46         inputPanel_1020.add(new JLabel("Masukkan angka (pisahkan dengan koma:"));
47         inputPanel_1020.add(inputField_1020);
48         inputPanel_1020.add(setButton_1020);
49
50         // Panel array visual
51         panelArray_1020 = new JPanel();
52         panelArray_1020.setLayout(new FlowLayout());
53
54         // Panel kontrol
55         JPanel controlPanel_1020 = new JPanel();
56         stepButton_1020 = new JButton("Langkah Selanjutnya");
57         resetButton_1020 = new JButton("Reset");
58         stepButton_1020.setEnabled(false);
59         controlPanel_1020.add(stepButton_1020);
60         controlPanel_1020.add(resetButton_1020);
61
62         // Area teks untuk log langkah-langkah
63         stepArea_1020 = new JTextArea(8, 60);
64         stepArea_1020.setEditable(false);
65         stepArea_1020.setFont(new Font("Monospaced", Font.PLAIN, 14));
66         JScrollPane scrollPane_1020 = new JScrollPane(stepArea_1020);
67
68         // Tambahkan panel ke frame
69         add(inputPanel_1020, BorderLayout.NORTH);
70         add(panelArray_1020, BorderLayout.CENTER);
71         add(controlPanel_1020, BorderLayout.SOUTH);
72         add(scrollPane_1020, BorderLayout.EAST);
73
74         // Event Set Array
75         setButton_1020.addActionListener(e -> setArrayFromInput_1020());
76
77         // Event Langkah Selanjutnya
78         stepButton_1020.addActionListener(e -> performStep_1020());
79
80         // Event Reset
81         resetButton_1020.addActionListener(e -> reset_1020());
82     }
83
84     // --- Tambahan dari foto pertama ---
85     private void setArrayFromInput_1020() {
86         String text_1020 = inputField_1020.getText().trim();
87         if (text_1020.isEmpty()) return;
88         String[] parts_1020 = text_1020.split(",");
89         array_1020 = new int[parts_1020.length];
90         try {
91             for (int k_1020 = 0; k_1020 < parts_1020.length; k_1020++) {
92                 array_1020[k_1020] = Integer.parseInt(parts_1020[k_1020].trim());
93             }
94         } catch (NumberFormatException e) {
95             JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan dengan koma",
96                 "Error", JOptionPane.ERROR_MESSAGE);
97             return;
98         }
99         i_1020 = 0;
100         stepCount_1020 = 1;
101         sorting_1020 = true;
102         stepButton_1020.setEnabled(true);
103         stepArea_1020.setText("");
104         panelArray_1020.removeAll();
105         labelArray_1020 = new JLabel[array_1020.length];
106         for (int k_1020 = 0; k_1020 < array_1020.length; k_1020++) {
107             labelArray_1020[k_1020] = new JLabel(String.valueOf(array_1020[k_1020]));
108             labelArray_1020[k_1020].setFont(new Font("Arial", Font.BOLD, 24));
109             labelArray_1020[k_1020].setOpaque(true);
110             labelArray_1020[k_1020].setBackground(Color.WHITE);
111             labelArray_1020[k_1020].setBorder(BorderFactory.createLineBorder(Color.BLACK));
112             labelArray_1020[k_1020].setPreferredSize(new Dimension(50, 50));
113             labelArray_1020[k_1020].setHorizontalAlignment(SwingConstants.CENTER);
114             panelArray_1020.add(labelArray_1020[k_1020]);
115         }
116         panelArray_1020.revalidate();

```

```

114         panelArray_1020.add(labelArray_1020[k_1020]);
115     }
116     panelArray_1020.revalidate();
117     panelArray_1020.repaint();
118 }
119
120 private void performStep_1020() {
121     if (i_1020 < array_1020.length && sorting_1020) {
122         int key_1020 = array_1020[i_1020];
123         j_1020 = i_1020 - 1;
124
125         StringBuilder stepLog_1020 = new StringBuilder();
126         stepLog_1020.append("Langkah ").append(stepCount_1020).
127         append(": Memasukkan ").append(key_1020).append("\n");
128
129         while (j_1020 >= 0 && array_1020[j_1020] > key_1020) {
130             array_1020[j_1020 + 1] = array_1020[j_1020];
131             j_1020--;
132         }
133         array_1020[j_1020 + 1] = key_1020;
134
135         updateLabels_1020();
136         stepLog_1020.append("Hasil: ").append(arrayToString_1020(array_1020)).append("\n\n");
137         stepArea_1020.append(stepLog_1020.toString());
138
139         i_1020++;
140         stepCount_1020++;
141     }
142
143     if (i_1020 == array_1020.length) {
144         sorting_1020 = false;
145         stepButton_1020.setEnabled(false);
146         JOptionPane.showMessageDialog(this, "Sorting selesai!");
147     }
148 }
149
150 // --- Tambahan dari foto kedua & ketiga ---
151 private void updateLabels_1020() {
152     for (int k_1020 = 0; k_1020 < array_1020.length; k_1020++) {
153         labelArray_1020[k_1020].setText(String.valueOf(array_1020[k_1020]));
154     }
155 }
156
157 private void resetHighlights_1020() {
158     for (JLabel label : labelArray_1020) {
159         label.setBackground(Color.WHITE);
160     }
161 }
162
163 private void reset_1020() {
164     inputField_1020.setText("");
165     panelArray_1020.removeAll();
166     panelArray_1020.revalidate();
167     panelArray_1020.repaint();
168     stepArea_1020.setText("");
169     stepButton_1020.setEnabled(false);
170     sorting_1020 = false;
171     i_1020 = 1;
172     stepCount_1020 = 1;
173 }
174
175 private String arrayToString_1020(int[] arr_1020) {
176     StringBuilder sb_1020 = new StringBuilder();
177     for (int k_1020 = 0; k_1020 < arr_1020.length; k_1020++) {
178         sb_1020.append(arr_1020[k_1020]);
179
180         if (k_1020 < arr_1020.length - 1) sb_1020.append(", ");
181     }
182     return sb_1020.toString();
183 }
184
185 public static void main(String[] args) {
186     javax.swing.SwingUtilities.invokeLater(() -> {
187         BubleSortGUI_2511531020 gui_1020 = new BubleSortGUI_2511531020();
188         gui_1020.setVisible(true);
189     });
190 }
191

```

Gambar 2.14 Class 4 “InsertionSortGUI_2511531020”

4. Analisis Langkah

a. Struktur Kelas & GUI

- Kamu membuat kelas BubleSortGUI_2511531020 yang extends JFrame.

- Ada panel input (untuk masukkan array angka), panel visual (menampilkan array dalam bentuk label), panel kontrol (tombol langkah & reset), dan area teks (log langkah-langkah sorting).
 - Semua komponen GUI sudah diatur dengan BorderLayout.
- b. Input Array
- Method `setArrayFromInput_1020()` membaca input dari `TextField`, memisahkan dengan koma, lalu mengubah ke array `int`.
 - Label dibuat untuk setiap elemen array, ditampilkan di panel dengan border dan background putih.
 - Tombol "Langkah Selanjutnya" diaktifkan setelah input valid.
- c. Proses Sorting (Step by Step)
- Method `performStep_1020()` menjalankan Insertion Sort, bukan Bubble Sort (walau nama kelasnya `BubbleSortGUI`).
 - Variabel `i_1020` digunakan untuk melacak posisi elemen yang sedang dimasukkan.
 - Elemen `key_1020` dibandingkan dengan elemen sebelumnya (`j_1020`), lalu digeser jika lebih besar.
 - Setelah satu langkah selesai, array diperbarui dan ditampilkan ulang di label.
 - Log langkah ditulis ke `stepArea_1020` dengan format:

Code

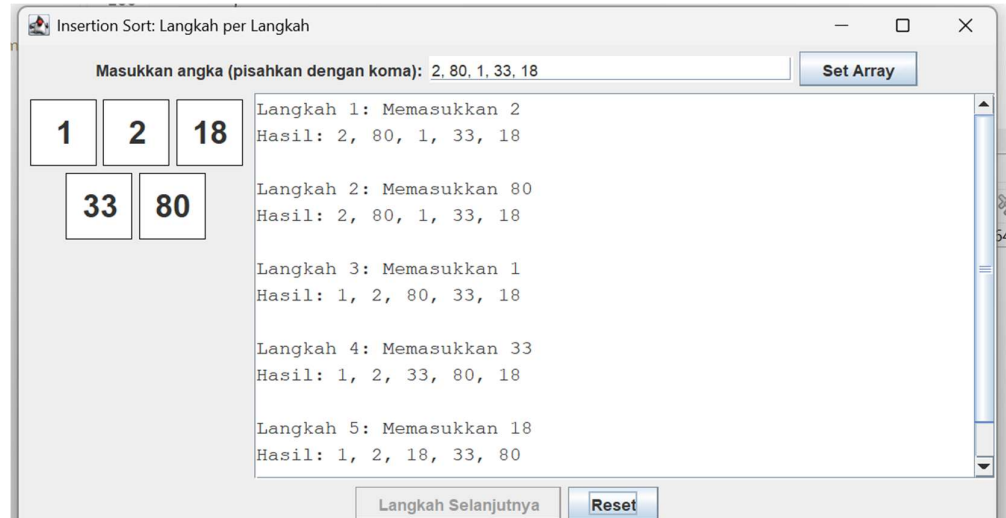
Langkah X: Memasukkan Y

Hasil: [array setelah langkah]

- Jika semua elemen sudah diproses (`i_1020 == array.length`), sorting selesai dan tombol langkah dinonaktifkan.
- d. Reset
- Method `reset_1020()` menghapus input, panel array, dan log langkah.
 - Tombol langkah dinonaktifkan, variabel direset ke kondisi awal.
- e. Main Program
- `main()` menjalankan GUI dengan `SwingUtilities.invokeLater()`.

- Frame ditampilkan dengan ukuran 750x400

5. Output

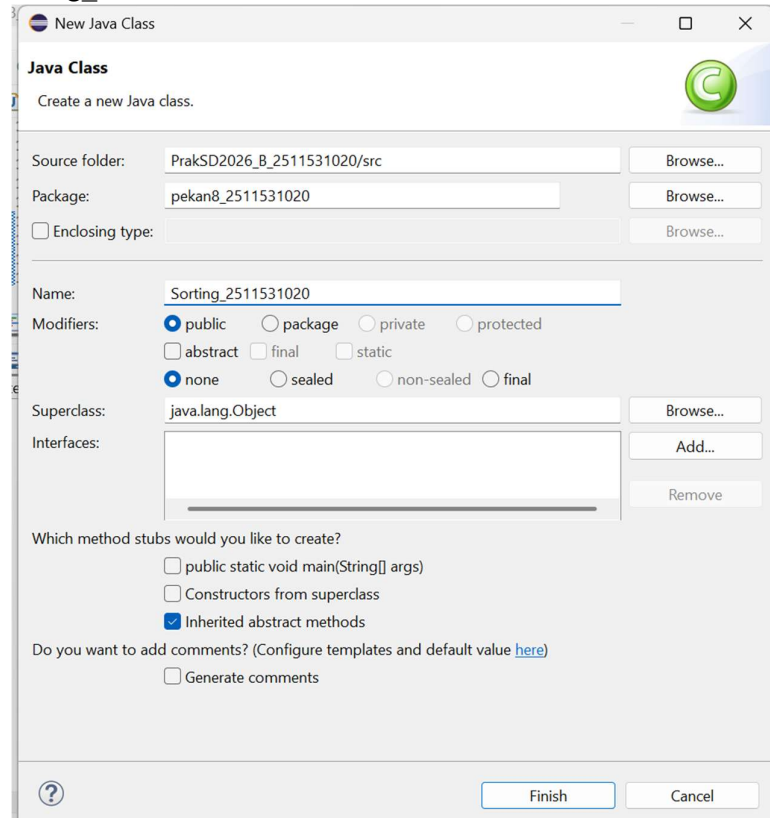


Gambar 2.15 Output

2.3 Tugas Praktikum

2.3.1 Class 1 “Sorting_2511531020”

1. Membuat Nama package pekan8_2511531020 dan class Sorting_2511531020



Gambar 2.16 package pekan8_2511531020 dan class Sorting_2511531020

2. Klik Finish
3. Kode Program

```
2. package pekan8_2511531020;
3.
4. import java.util.Scanner;
5.
6. // Kelas Lagu
7. class Lagu {
8.     String judul;
9.     String penyanyi;
10.    int durasi;
11.
12.    Lagu(String judul, String penyanyi, int durasi)
13.    {
14.        this.judul = judul;
15.        this.penyanyi = penyanyi;
```

```

15.         this.durasi = durasi;
16.     }
17. }
18.
19. // Kelas Sorting_NIM
20. public class Sorting_2511531020 {
21.
22.     Lagu[] dataLagu_1020 = new Lagu[20];
23.     int jumlahData_1020 = 0;
24.
25.     // Input minimal 7 lagu
26.     public void inputData_1020() {
27.         dataLagu_1020[jumlahData_1020++] = new
Lagu("Mio Cristo Piange Diamanti", "Artis A", 270);
28.         dataLagu_1020[jumlahData_1020++] = new
Lagu("La Rumba Del Perdon", "Artis B", 252);
29.         dataLagu_1020[jumlahData_1020++] = new
Lagu("La Perla", "Artis C", 196);
30.         dataLagu_1020[jumlahData_1020++] = new
Lagu("Song D", "Artis D", 310);
31.         dataLagu_1020[jumlahData_1020++] = new
Lagu("Song E", "Artis E", 180);
32.         dataLagu_1020[jumlahData_1020++] = new
Lagu("Song F", "Artis F", 220);
33.         dataLagu_1020[jumlahData_1020++] = new
Lagu("Song G", "Artis G", 205);
34.     }
35.
36.     // Tampil data
37.     public void tampilData_1020() {
38.         for (int i = 0; i < jumlahData_1020; i++) {
39.             System.out.println((i + 1) + ". " +
dataLagu_1020[i].judul + " - " +
dataLagu_1020[i].durasi + " detik");
40.         }
41.     }
42.
43.     // =====
44.     // Quick Sort (Durasi Ascending)
45.     // =====
46.     public void quickSort_1020(int low, int high) {
47.         if (low < high) {
48.             int pi = partition_1020(low, high);
49.             quickSort_1020(low, pi - 1);
50.             quickSort_1020(pi + 1, high);
51.         }
52.     }
53.
54.     private int partition_1020(int low, int high) {
55.         int pivot = dataLagu_1020[high].durasi;
56.         int i = (low - 1);
57.         for (int j = low; j < high; j++) {
58.             if (dataLagu_1020[j].durasi <= pivot) {
59.                 i++;
60.                 Lagu temp = dataLagu_1020[i];
61.                 dataLagu_1020[i] = dataLagu_1020[j];
62.                 dataLagu_1020[j] = temp;

```

```

63.     }
64.   }
65.   Lagu temp = dataLagu_1020[i + 1];
66.   dataLagu_1020[i + 1] = dataLagu_1020[high];
67.   dataLagu_1020[high] = temp;
68.   return i + 1;
69. }
70.
71. // =====
72. // Merge Sort (Judul A-Z)
73. // =====
74. public void mergeSort_1020(int left, int right)
75. {
76.     if (left < right) {
77.         int mid = (left + right) / 2;
78.         mergeSort_1020(left, mid);
79.         mergeSort_1020(mid + 1, right);
80.         merge_1020(left, mid, right);
81.     }
82. }
83. private void merge_1020(int left, int mid, int
84. right) {
85.     int n1 = mid - left + 1;
86.     int n2 = right - mid;
87.     Lagu[] L = new Lagu[n1];
88.     Lagu[] R = new Lagu[n2];
89.
90.     for (int i = 0; i < n1; i++) L[i] =
91. dataLagu_1020[left + i];
92.     for (int j = 0; j < n2; j++) R[j] =
93. dataLagu_1020[mid + 1 + j];
94.
95.     int i = 0, j = 0, k = left;
96.     while (i < n1 && j < n2) {
97.         if
98. (L[i].judul.compareToIgnoreCase(R[j].judul) <= 0) {
99.             dataLagu_1020[k++] = L[i++];
100.        } else {
101.            dataLagu_1020[k++] = R[j++];
102.        }
103.    }
104.    while (i < n1) dataLagu_1020[k++] =
105. L[i++];
106.    while (j < n2) dataLagu_1020[k++] =
107. R[j++];
108. }
109. // =====
110. // Shell Sort (Judul A-Z)
111. // =====
112. public void shellSort_1020() {
113.     int n = jumlahData_1020;
114.     for (int gap = n / 2; gap > 0; gap /=
115. 2) {
116.         for (int i = gap; i < n; i++) {

```

```

112.             Lagu temp = dataLagu_1020[i];
113.             int j;
114.             for (j = i; j >= gap &&
dataLagu_1020[j -
gap].judul.compareToIgnoreCase(temp.judul) > 0; j -=
gap) {
115.                 dataLagu_1020[j] =
dataLagu_1020[j - gap];
116.             }
117.             dataLagu_1020[j] = temp;
118.         }
119.     }
120. }
121.
122. // =====
123. // Main Program
124. // =====
125. public static void main(String[] args) {
126.     Sorting_2511531020 s = new
Sorting_2511531020();
127.     s.inputData_1020();
128.
129.     Scanner sc = new Scanner(System.in);
130.     System.out.println("=== Sorting
Playlist NIM: 2511531020 ===");
131.     System.out.println("Pilih Algoritma
(1=Shell, 2=Quick, 3=Merge): ");
132.     int pilihan = sc.nextInt();
133.
134.     System.out.println("\nData Sebelum
Sorting:");
135.     s.tampilData_1020();
136.
137.     switch (pilihan) {
138.         case 1:
139.             s.shellSort_1020();
140.             System.out.println("\nData
Setelah Shell Sort (Judul A-Z):");
141.             break;
142.         case 2:
143.             s.quickSort_1020(0,
s.jumlahData_1020 - 1);
144.             System.out.println("\nData
Setelah Quick Sort (Durasi Asc):");
145.             break;
146.         case 3:
147.             s.mergeSort_1020(0,
s.jumlahData_1020 - 1);
148.             System.out.println("\nData
Setelah Merge Sort (Judul A-Z):");
149.             break;
150.         default:
151.             System.out.println("Pilihan
tidak valid!");
152.             return;
153.     }
154.

```

```

155.         s.tampilData_1020();
156.     }
157. }

```

Gambar 2.17 Class “Sorting_2511531020”

4. Analisis Langkah

a. Struktur Kelas

- Ada dua kelas:
 - class Lagu → menyimpan data lagu (judul, penyanyi, durasi).
 - class Sorting_2511531020 → berisi array lagu, input data, algoritma sorting, dan main().
- Input Data
- Method inputData_1020() mengisi array dataLagu_1020 dengan 7 lagu (judul, penyanyi, durasi).
- Sesuai aturan soal: minimal 7 data, maksimal 20.

b. Tampil Data

- Method tampilData_1020() menampilkan isi array ke console.
- Format: nomor urut, judul lagu, dan durasi.

c. Algoritma Sorting

- Quick Sort (quickSort_1020)
 - Mengurutkan berdasarkan durasi ascending.
 - Fungsi partition_1020 memilih pivot, lalu menukar posisi elemen agar lebih kecil di kiri, lebih besar di kanan.
- Merge Sort (mergeSort_1020)
 - Mengurutkan berdasarkan judul A–Z.
 - Fungsi merge_1020 membagi array jadi dua, lalu menggabungkan kembali dengan urutan alfabet.
- Shell Sort (shellSort_1020)
 - Mengurutkan berdasarkan judul A–Z.

- Menggunakan gap yang mengecil untuk membandingkan dan menukar posisi judul.

d. Main Program

- Membuat objek Sorting_2511531020.
- Memanggil inputData_1020() untuk mengisi data.
- Menampilkan menu pilihan algoritma:

Code:

Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):

- User memasukkan angka (1, 2, atau 3).
- Program menampilkan data sebelum sorting, lalu menjalankan algoritma sesuai pilihan.
- Setelah itu, program menampilkan data sesudah sorting.

5. Output

- a. Output tersebut memperlihatkan hasil eksekusi program saat memilih algoritma Quick Sort. Program menampilkan daftar lagu sebelum diurutkan, lalu setelah proses Quick Sort selesai, playlist tersusun berdasarkan durasi dari yang terpendek (180 detik, “Song E”) hingga terpanjang (310 detik, “Song D”)

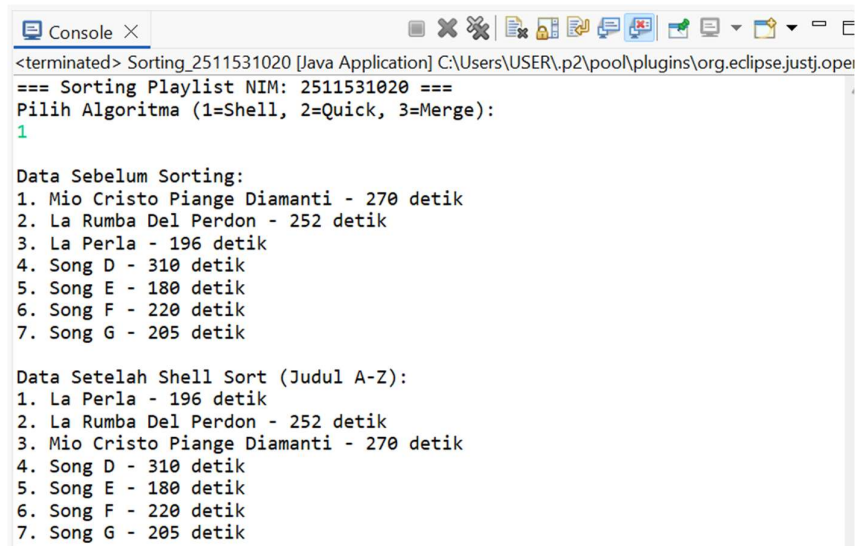
```
<terminated> Sorting_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.open...
=== Sorting Playlist NIM: 2511531020 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
2

Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. Song D - 310 detik
5. Song E - 180 detik
6. Song F - 220 detik
7. Song G - 205 detik

Data Setelah Quick Sort (Durasi Asc):
1. Song E - 180 detik
2. La Perla - 196 detik
3. Song G - 205 detik
4. Song F - 220 detik
5. La Rumba Del Perdon - 252 detik
6. Mio Cristo Piange Diamanti - 270 detik
7. Song D - 310 detik
```

Gambar 2.18 Output

- b. Output tersebut menunjukkan hasil program saat memilih algoritma Shell Sort (pilihan 1). Pertama ditampilkan daftar lagu sebelum diurutkan, lalu setelah proses Shell Sort selesai playlist tersusun berdasarkan judul lagu dari A–Z. Hasilnya terlihat “La Perla” muncul di urutan pertama, diikuti “La Rumba Del Perdon”, kemudian “Mio Cristo Piange Diamanti”, dan seterusnya hingga “Song G” di akhir, sehingga urutan berubah sesuai alfabet judul lagu.

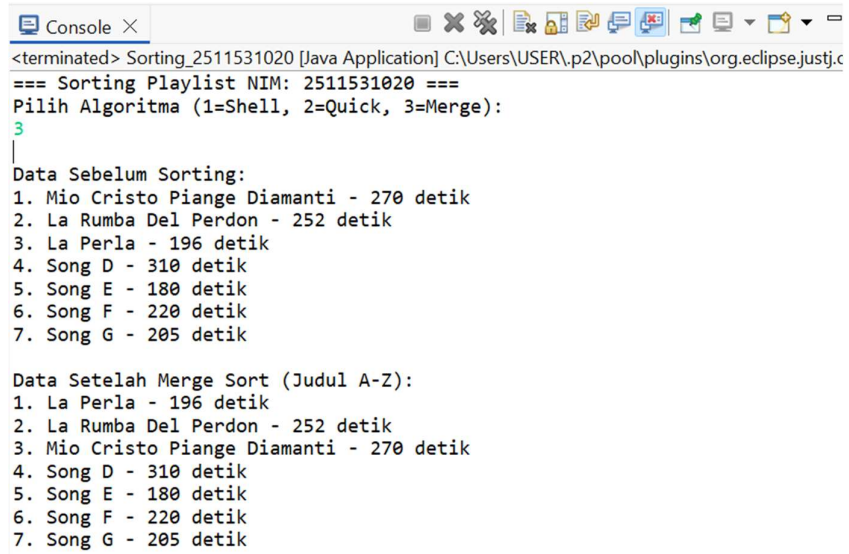


```
<terminated> Sorting_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.open...
=== Sorting Playlist NIM: 2511531020 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
1
Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. Song D - 310 detik
5. Song E - 180 detik
6. Song F - 220 detik
7. Song G - 205 detik

Data Setelah Shell Sort (Judul A-Z):
1. La Perla - 196 detik
2. La Rumba Del Perdon - 252 detik
3. Mio Cristo Piange Diamanti - 270 detik
4. Song D - 310 detik
5. Song E - 180 detik
6. Song F - 220 detik
7. Song G - 205 detik
```

Gambar 2.19 Output

- c. Output tersebut menunjukkan hasil program saat memilih algoritma Merge Sort (pilihan 3). Pertama ditampilkan daftar lagu sebelum diurutkan, lalu setelah proses Merge Sort selesai playlist tersusun berdasarkan judul lagu dari A–Z. Hasilnya “La Perla” muncul di urutan pertama, diikuti “La Rumba Del Perdon”, kemudian “Mio Cristo Piange Diamanti”, dan seterusnya hingga “Song G” di akhir, sehingga urutan berubah sesuai alfabet judul lagu.



```
<terminated> Sorting_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\org.eclipse.justj.c
=== Sorting Playlist NIM: 2511531020 ===
Pilih Algoritma (1=Shell, 2=Quick, 3=Merge):
3
|
Data Sebelum Sorting:
1. Mio Cristo Piange Diamanti - 270 detik
2. La Rumba Del Perdon - 252 detik
3. La Perla - 196 detik
4. Song D - 310 detik
5. Song E - 180 detik
6. Song F - 220 detik
7. Song G - 205 detik

Data Setelah Merge Sort (Judul A-Z):
1. La Perla - 196 detik
2. La Rumba Del Perdon - 252 detik
3. Mio Cristo Piange Diamanti - 270 detik
4. Song D - 310 detik
5. Song E - 180 detik
6. Song F - 220 detik
7. Song G - 205 detik
```

Gambar 2.20 Output

BAB III

KESIMPULAN

Setiap algoritma sorting memiliki karakteristik, kelebihan, dan kekurangan masing-masing; Quick Sort efisien untuk data besar dengan pemilihan pivot yang tepat, Merge Sort stabil dengan kompleksitas $O(n \log n)$, Shell Sort sederhana namun cukup cepat, sedangkan Bubble/Insertion Sort lebih cocok untuk pembelajaran dasar karena mudah dipahami; selain itu, visualisasi GUI memberikan pemahaman interaktif mengenai proses pengurutan, dan penerapan sorting pada playlist lagu menunjukkan bahwa algoritma tidak hanya relevan untuk data numerik tetapi juga dapat digunakan pada data teks, sehingga praktikum ini berhasil memberikan gambaran komprehensif mengenai teori, implementasi, serta aplikasi nyata algoritma sorting dalam pemrograman Java.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
- [2] R. Lafore, *Data Structures and Algorithms in Java*, 2nd ed. Indianapolis: Sams Publishing, 2003.
- [3] E. Horowitz, S. Sahni, and D. Mehta, *Fundamentals of Data Structures in C++*, 2nd ed. Summit, NJ: Silicon Press, 2007.
- [4] Oracle, "Java SE Documentation," Oracle Help Center. [Online]. Available: <https://docs.oracle.com/javase> [Accessed: Jun. 5, 2026].
- [5] A. Aziz Jabar and A. S. Anas, "Aplikasi Belajar Interaktif Algoritma Sorting Berbasis Desktop," *JTIM: Jurnal Teknologi Informasi dan Multimedia*, vol. 1, no. 1, pp. 23–29, May 2019. doi: 10.35746/jtim.v1i1.10